TIMO 포팅 매뉴얼



1. 구성 및 사전 준비

구성	기술	배포 대상
팬 앱	Flutter	Android APK
게이트 앱	Flutter	Android APK
백엔드	Spring Boot · Java 21	Docker / EC2
블록체인	Hardhat · Solidity	Node.js 서버
데이터	MySQL · Redis	Amazon RDS · ElastiCache

항목	필수 버전 / 도구
개발 PC	Git LFS, Node.js 20+, npm, Flutter SDK, JDK 21
서버	Docker, AWS CLI, ECR 접근 권한
AWS	EC2, RDS MySQL, ElastiCache Redis, S3, CloudFront, ALB, Route 53, ACM, Secrets Manager

2. 외부 서비스

서비스	용도
AWS	서버·DB·이미지·도메인
Firebase	앱 푸시
Solapi	SMS
NAVER CAPTCHA	자동입력 방지
Hardhat	NFT 발급·검증

3. 소스 준비

```
git clone <REPOSITORY_URL>
cd S15P21D206
git lfs install --local
git lfs pull
git lfs fsck
```

4. AWS 구성

4.1 구축 순서 및 생성 대상

대상	필수 설정
네트워크	VPC, ALB 용 공용 서브넷 2 개, 백엔드용 서브넷, NAT
EC2	Docker 설치, ECR Pull·Secrets Manager GetSecretValue·SSM 권한 역할 연결
RDS MySQL	DB 생성 후 infra/bootstrap/init.sql 적용
ElastiCache Redis	Redis 7.1, TLS/AUTH, endpoint 확인
S3 · CloudFront	public/private 이미지 버킷, public 버킷 CloudFront 배포
ALB · 도메인	Target Group: EC2 8080, Health Check: /actuator/health, DNS: api.<DOMAIN>
Secrets Manager	backend 환경변수 JSON 저장

1. VPC 와 서브넷을 생성한다. ALB 용 공용 서브넷 2 개와 백엔드 EC2 용 서브넷을 만들고, EC2 가 필요한 AWS 서비스에 연결될 수 있도록 NAT 를 구성한다.

2. RDS MySQL 을 생성한다. DB 이름과 마스터 계정을 정한 뒤, DB 를 만든 다음 infra/bootstrap/init.sql 을 적용한다. 생성 후 endpoint, 포트, DB 이름, 계정, 비밀번호를 기록한다.

```
mysql -h <RDS_ENDPOINT> -P 3306 -u <MASTER_USER> -p -e "CREATE DATABASE timo CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;"
mysql -h <RDS_ENDPOINT> -P 3306 -u <MASTER_USER> -p timo < infra/bootstrap/init.sql
```

3. ElastiCache Redis 를 생성한다. Redis 7.1 을 선택하고 TLS/AUTH 를 설정한다. 생성 후 endpoint, 포트, AUTH 토큰을 기록한다.

4. S3 버킷을 생성한다. 공개 이미지용 버킷과 비공개 이미지용 버킷을 만들고, 공개 이미지용 버킷을 원본으로 CloudFront 배포를 생성한다. 버킷명과 CloudFront 도메인을 기록한다.

5. Secrets Manager 에 백엔드 설정을 저장한다. 4.2 환경변수 예시의 CHANGE_ME 값을 실제 값으로 바꿔 JSON 또는 backend.env 형식으로 저장한다.

6. ECR 리포지토리를 생성한다. 리포지토리 URI 를 기록하고, 배포 시 백엔드 Docker 이미지를 이 리포지토리에 푸시한다.

7. EC2 를 생성한다. Ubuntu 22.04 를 사용하고 Docker, AWS CLI, Node.js 20+, jq 를 설치한다. ECR 이미지 조회·Secrets Manager 조회·SSM 접속·S3 객체 조회/업로드/삭제 권한이 있는 IAM 역할을 연결한다. Amazon Linux 를 사용할 경우 jq 는 sudo dnf install -y jq 로 설치한다.

8. ALB 를 생성한다. Target Group 은 백엔드 EC2 의 8080 포트로 만들고, Health Check 경로는 /actuator/health 로 설정한다.

9. 도메인과 인증서를 연결한다. ACM 인증서를 발급하고 Route 53 에 api.<DOMAIN> 레코드를 추가해 ALB 로 연결한다.

10. 백엔드 배포 후 https://api.<DOMAIN>/actuator/health 가 정상 응답하는지 확인한다.

4.2 환경변수 예시

Secrets Manager JSON 또는 /etc/timo/backend.env 에 실제 값으로 입력합니다.

```
SPRING_PROFILES_ACTIVE=prod
DB_URL=jdbc:mysql://CHANGE_ME:3306/CHANGE_ME?connectionTimeZone=Asia/Seoul
DB_USERNAME=CHANGE_ME
DB_PASSWORD=CHANGE_ME
REDIS_HOST=CHANGE_ME
REDIS_PORT=6379
REDIS_PASSWORD=CHANGE_ME
TRACING_OTLP_ENDPOINT=http://<JAEGER_PRIVATE_IP>:4318/v1/traces
JWT_SECRET=CHANGE_ME
ENTRY_PASS_SECRET=CHANGE_ME
AWS_ACCESS_KEY_ID=CHANGE_ME
AWS_SECRET_ACCESS_KEY=CHANGE_ME
AWS_REGION=ap-northeast-2
S3_PUBLIC_BUCKET_NAME=CHANGE_ME
IMAGE_BASE_URL=https://CHANGE_ME
WALLET_MASTER_KEY=CHANGE_ME
ADMIN_PRIVATE_KEY=CHANGE_ME
BLOCKCHAIN_RPC_URL=http://CHANGE_ME:8545
SOLAPI_API_KEY=CHANGE_ME
SOLAPI_API_SECRET=CHANGE_ME
SOLAPI_SENDER_PHONE=CHANGE_ME
SMS_LIVE_TEST_TO=CHANGE_ME
NAVER_CAPTCHA_CLIENT_ID=CHANGE_ME
NAVER_CAPTCHA_CLIENT_SECRET=CHANGE_ME
FIREBASE_MESSAGING_ENABLED=true
PUSH_CHANNEL_GENERAL=timo_general_notifications
GOOGLE_APPLICATION_CREDENTIALS=/run/secrets/firebase-service-account.json
SEED_ORGANIZER_PHONE=CHANGE_ME
SEED_ORGANIZER_NAME=CHANGE_ME
SEED_ORGANIZER_NICKNAME=CHANGE_ME
SEED_ADMIN_PHONE=CHANGE_ME
SEED_ADMIN_NAME=CHANGE_ME
SEED_ADMIN_NICKNAME=CHANGE_ME
SEED_ADMIN_USERNAME=CHANGE_ME
SEED_ADMIN_PASSWORD=CHANGE_ME
```

4.3 백엔드 블록체인 URL 반영

backend/timo/src/main/resources/application.yml 을 아래처럼 변경한 뒤 백엔드 이미지를 다시 빌드합니다.

```
blockchain:
  rpc-url: ${BLOCKCHAIN_RPC_URL:http://network.ticketmoa.site:8545}
  admin-private-key: ${ADMIN_PRIVATE_KEY:}
```

5. 배포

5.1 Hardhat 서버

체인 ID 는 31337 입니다. Hardhat 을 먼저 상시 실행하고, 백엔드의 BLOCKCHAIN_RPC_URL 에는 Hardhat 서버의 주소를 입력합니다.

```
cd blockchain
npm ci
export ADMIN_PRIVATE_KEY=0xCHANGE_ME
npx hardhat node --hostname 0.0.0.0
```

Hardhat 프로세스는 백엔드보다 먼저 실행한 뒤 계속 유지한다. Hardhat 이 재시작되면 새 체인이므로, 백엔드를 실행하기 전에 아래 deploy 명령을 한 번 다시 실행한다.

```
cd blockchain
export ADMIN_PRIVATE_KEY=0xCHANGE_ME
npm run deploy -- --network localhost
```

Hardhat 실행·컨트랙트 배포·백엔드에는 같은 ADMIN_PRIVATE_KEY 를 사용한다. Hardhat 프로세스가 유지되는 동안에는 deploy 를 다시 실행하지 않는다.

5.2 백엔드 이미지 빌드 및 ECR 푸시

```
cd backend/timo
./gradlew clean bootJar
docker build -t timo-backend .
aws ecr get-login-password --region <AWS_REGION> | docker login --username AWS --password-stdin
<AWS_ACCOUNT_ID>.dkr.ecr.<AWS_REGION>.amazonaws.com
docker tag timo-backend:latest <ECR_REPOSITORY_URI>:latest
docker push <ECR_REPOSITORY_URI>:latest
```

5.3 EC2 실행

```
aws secretsmanager get-secret-value --secret-id <SECRET_ID> --query SecretString --output text > /tmp/timo-
secret.json
sudo install -d -m 0700 /etc/timo
sudo apt-get update && sudo apt-get install -y jq
jq -r 'to_entries[] | "₩(.key)=₩(.value)"' /tmp/timo-secret.json | sudo tee /etc/timo/backend.env > /dev/null
sudo chmod 600 /etc/timo/backend.env
aws ecr get-login-password --region <AWS_REGION> | docker login --username AWS --password-stdin
<AWS_ACCOUNT_ID>.dkr.ecr.<AWS_REGION>.amazonaws.com
docker pull <ECR_REPOSITORY_URI>:latest
docker rm -f timo-backend 2>/dev/null || true
docker run -d --name timo-backend --restart unless-stopped --env-file /etc/timo/backend.env -v
/etc/timo/firebase-service-account.json:/run/secrets/firebase-service-account.json:ro -p 8080:8080
<ECR_REPOSITORY_URI>:latest
```

Firebase 푸시를 사용할 경우 docker run 전에 서비스 계정 JSON 을 /etc/timo/firebase-service-account.json 에 배치하고 권한을 제한한다.

sudo install -m 600 <FIREBASE_SERVICE_ACCOUNT_JSON_PATH> /etc/timo/firebase-service-account.json

backend.env 에는 GOOGLE_APPLICATION_CREDENTIALS=/run/secrets/firebase-service-account.json 을 입력한다.
Firebase 를 사용하지 않을 경우 FIREBASE_MESSAGING_ENABLED=false 로 설정하고 docker run 명령에서 -v /etc/timo/firebase-service-account.json:/run/secrets/firebase-service-account.json:ro 옵션을 제거한다.

5.4 앱 빌드

```
# 팬 앱
cd frontend
flutter pub get
flutter build apk --release --dart-define=API_BASE_URL=https://api.<DOMAIN>

# 게이트 앱
cd gate
flutter pub get
flutter build apk --release --dart-define=API_BASE_URL=https://api.<DOMAIN>
```

팬 앱은 새 Firebase 프로젝트에서 Android 앱 패키지명 com.ssafy.timo 를 등록한 뒤 받은 frontend/android/app/google-services.json 을 배치한다.

6. 검증

순서	확인
1	https://api.<DOMAIN>/actuator/health 응답 확인
2	팬 앱: 가입·로그인·공연 조회·결제·티켓 확인
3	게이트 앱: 로그인·QR 스캔·입장 처리
4	SMS, CAPTCHA, 푸시, 이미지 업로드 확인